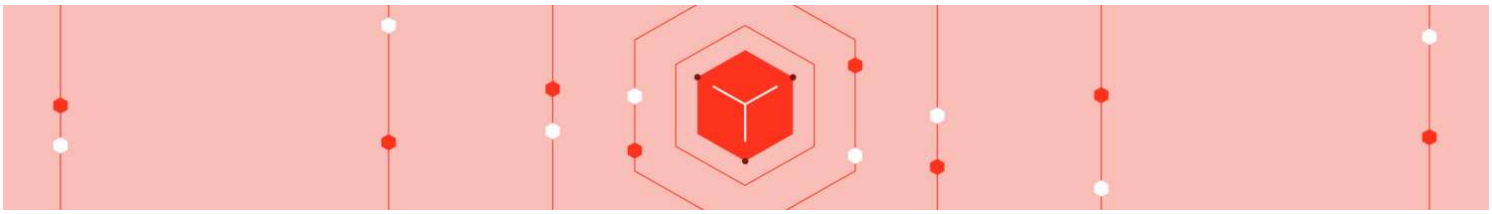


Introducing Delta Time Travel for Large Scale Data Lakes

by Burak Yavuz and Prakash Chockalingam

February 4, 2019 in Company Blog

Share this post

Get an early preview of O'Reilly's new ebook for the step-by-step guidance you need to start using Delta Lake.

Data versioning for reproducing experiments, rolling back, and auditing data

We are thrilled to introduce time travel capabilities in Databricks Delta Lake, the next-gen unified analytics engine built on top of Apache Spark, for all of our users. With this new feature, Delta automatically versions the big data that you store in your data lake, and you can access any historical version of that data. This temporal data management simplifies your data pipeline by making it easy to audit, roll back data in case of accidental bad writes or deletes, and reproduce experiments and reports. Your organization can finally standardize on a clean, centralized, versioned big data repository in your own cloud storage for your analytics.

Common Challenges with Changing Data

- **Audit data changes**: Auditing data changes is critical from both in terms of data compliance as well as simple debugging to understand how data has changed over time. Organizations moving from traditional data systems to big data technologies and the cloud struggle in such scenarios.
- **Reproduce experiments & reports**: During model training, data scientists run various experiments with different parameters on a given set of data. When scientists revisit their experiments after a period of time to reproduce the models, typically the source data has been modified by upstream pipelines. Lot of times, they are caught unaware by such upstream data changes and hence struggle to reproduce their experiments. Some scientists and organizations engineer best practices by creating multiple copies of the data, leading to increased storage costs. The same is true for analysts generating reports.
- **Rollbacks**: Data pipelines can sometimes write bad data for downstream consumers. This can happen because of issues ranging from infrastructure instabilities to messy data to bugs in the pipeline. For pipelines that do simple appends to directories or a table, rollbacks can easily be addressed by date-based partitioning. With updates and deletes, this can become very complicated, and data engineers typically have to engineer a complex pipeline to deal with such scenarios.

Delta's time travel capabilities simplify building data pipelines for the above use cases. As you write into a Delta table or directory, every operation is automatically versioned. You can access the different versions of the data two different ways:

1. Using a timestamp

Scala syntax:

You can provide the timestamp or date string as an option to DataFrame reader:

```
val df = spark.read
  .format("delta")
  .option("timestampAsOf", "2019-01-01")
  .load("/path/to/my/table")
```

In Python:

```
df = spark.read \
  .format("delta") \
  .option("timestampAsOf", "2019-01-01") \
  .load("/path/to/my/table")
```

SQL syntax:

```
SELECT count(*) FROM my_table TIMESTAMP AS OF "2019-01-01"
SELECT count(*) FROM my_table TIMESTAMP AS OF date_sub(current_date(), 1)
SELECT count(*) FROM my_table TIMESTAMP AS OF "2019-01-01 01:30:00.000"
```

If the reader code is in a library that you don't have access to, and if you are passing input parameters to the library to read data, you can still travel back in time for a table by passing the timestamp in yyyyMMddHHmmssSSS format to the path:

```
val df = loadData(inputPath)

// Function in a library that you don't have access to
def loadData(inputPath : String) : DataFrame = {
  spark.read
    .format("delta")
    .load(inputPath)
}
```

```
inputPath = "/path/to/my/table@20190101000000000"
df = loadData(inputPath)

# Function in a library that you don't have access to
def loadData(inputPath):
  return spark.read \
    .format("delta") \
    .load(inputPath)
}
```

2. Using a version number

In Delta, every write has a version number, and you can use the version number to travel back in time as well.

Scala syntax:

```
val df = spark.read
  .format("delta")
  .option("versionAsOf", "5238")
  .load("/path/to/my/table")

val df = spark.read
  .format("delta")
  .load("/path/to/my/table@v5238")
```

Python syntax:

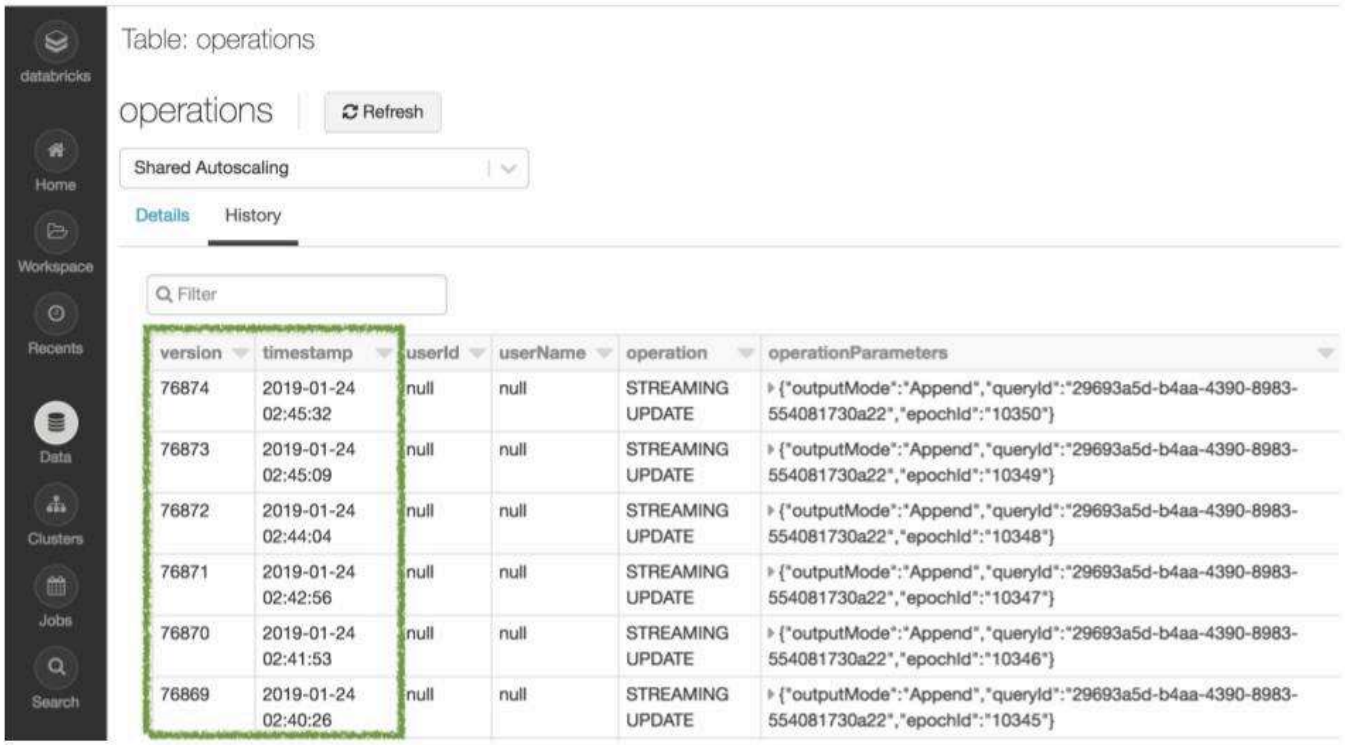
```
  .format("delta") \
  .option("versionAsOf", "5238") \
  .load("/path/to/my/table")

df = spark.read \
  .format("delta") \
  .load("/path/to/my/table@v5238")
```

SQL syntax:

```
SELECT count(*) FROM my_table VERSION AS OF 5238
SELECT count(*) FROM my_table@v5238
SELECT count(*) FROM delta.`/path/to/my/table@v5238`
```

Audit data changes

You can look at the history of table changes using the DESCRIBE HISTORY command or through the UI.



Time travel also plays an important role in machine learning and data science. Reproducibility of models and experiments is a key consideration for data scientists, because they often create 100s of models before they put one into production, and in that time-consuming process would like to go back to earlier models. However, because data management is often separate from data science tools, this is really hard to accomplish.

Databricks solves this reproducibility problem by integrating Delta's time-travel capabilities with MLflow, an open source platform for the machine learning lifecycle. For reproducible machine learning training, you can simply log a timestamped URL to the path as an MLflow parameter to track which version of the data was used for each training job. This enables you to go back to earlier settings and datasets to reproduce earlier models. You neither need to coordinate with upstream teams on the data nor worry about cloning data for different experiments. This is the power of Unified Analytics, whereby data science is closely married with data engineering.

Rollbacks

Time travel also makes it easy to do rollbacks in case of bad writes. For example, if your GDPR pipeline job had a bug that accidentally deleted user information, you can easily fix the pipeline:

```
INSERT INTO my_table
SELECT * FROM my_table TIMESTAMP AS OF date_sub(current_date(), 1)
WHERE userId = 111
```

You can also fix incorrect updates as follows:

```
MERGE INTO my_table target
USING my_table TIMESTAMP AS OF date_sub(current_date(), 1) source
ON source.userId = target.userId
WHEN MATCHED THEN UPDATE SET *
```

multiple downstream jobs

With AS OF queries, you can now pin the snapshot of a continuously updating Delta table for multiple downstream jobs. Consider a situation where a Delta table is being continuously updated, say every 15 seconds, and there is a downstream job that periodically reads from this Delta table and updates different destinations. In such scenarios, typically you want a consistent view of the source Delta table so that all destination tables reflect the same state. You can now easily handle such scenarios as follows:

```
version = spark.sql("SELECT max(version) FROM (DESCRIBE HISTORY my_table)").co

# Will use the latest version of the table for all operations below

data = spark.table("my_table@v%s" % version[0][0]

data.where("event_type = e1").write.jdbc("table1")
data.where("event_type = e2").write.jdbc("table2")
...
data.where("event_type = e10").write.jdbc("table10")
```

Queries for time series analytics made simple

Time travel also simplifies time series analytics. For example, if you want to find out how many new customers you added over the last week, your query could be a very simple one like this:

```
SELECT count(distinct userId) - (
SELECT count(distinct userId)
FROM my_table TIMESTAMP AS OF date_sub(current_date(), 7))
FROM my_table
```

Conclusion

Time travel in Delta improves developer productivity tremendously. It helps:

- Data scientists manage their experiments better
- Data engineers simplify their pipelines and roll back bad writes
- Data analysts do easy reporting

Organizations can finally standardize on a clean, centralized, versioned big data repository in their own cloud storage for analytics. We are thrilled to see what you will be able to accomplish with this new feature.

The feature is available as public preview for all users. Learn more about the feature. To see it in action, sign up for a free trial of Databricks.

Interested in the open source Delta Lake?
Visit the Delta Lake online hub to learn more, download the latest code and join the Delta Lake community.

Related posts

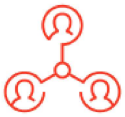
Introducing Delta Time Travel for Large Scale Data Lakes

February 4, 2019 by Burak Yavuz and Prakash Chockalingam in Company Blog

Get an early preview of O'Reilly's new ebook for the step-by-step guidance you need to start using Delta Lake . Data versioning for...

databricks

Login



Databricks Inc.
160 Spear Street, 13th Floor
San Francisco, CA 94105
1-866-330-0121

See Careers
at Databricks